

Fundación Fulgor

Diseño Digital Avanzado

GP01

Nombre	DNI
Corvalán, Abel Nicolás	41.220.050

Docentes	Ing. Pola, Ariel Cayuela, Pablo.
----------	-------------------------------------

Córdoba, República Argentina
30 de septiembre de 2026

Índice

1. Parte A — Propiedades del código	2
2. Parte B — Modelo de referencia en software	3
2.1. Caracterización del decodificador	3
2.2. Clase GaloisField en GF(2)	4
2.3. Modelado del encoder y decoder	5
3. Parte C — Implementación RTL	6
4. Codificador	6
4.1. Validación del encoder	8
4.2. Desarrollo y verificación de módulos comunes	8
4.3. Analisis: Posible modificacion del decoder	8
4.4. Implementación RTL del encoder	9
4.5. Resumen del encoder	9
4.6. Implementación RTL del decoder	9
4.7. Resumen del decoder	12
4.8. Validacion del decoder	12
4.9. Correcciones en el decoder	13
4.10. Integración del top level	14
4.11. Verificación del top level	16

1. Parte A — Propiedades del código

PROMPT: estoy haciendo un informe tecnico (para un curso de posgrado dedicado a disenio de RTL de codigos fec) y necesito dar formato a algunos pasos que realizamos en el trabajo. si te paso la idea general de los pasos realizados, podrias cambiar tiempos verbales y formatearlo como reporte tecnico?. Puedes darme el latex del los resultados?. No agregues modificaciones al tipo de letra en los textos, yo decido si agregar cosas como `textit`, `textbf`, etc

2. Parte B — Modelo de referencia en software

2.1. Caracterización del decodificador

Para cada peso entre 0 y 4, se recorre la totalidad de los patrones de error correspondientes a dicho peso. Estos vectores son generados en el golden model `golay_decoder.ipynb`.

Luego, estos vectores son procesados por un objeto decodificador para obtener el archivo de vectores `outputs/decoder/decoder_top_testing/codeword_with_errors.txt`.

Este archivo tiene la siguiente convención de vectores.

1. Palabra recibida	2. Mensaje	3. Máscara de error	4. Palabra corregida	5. Palabra incorregible
---------------------	------------	---------------------	----------------------	-------------------------

Tabla 1: Descripción de las columnas de salida del decodificador.

El script con el cual se genera el conjunto de vectores es el siguiente:

Listing 1: Generación de los vectores de prueba del decodificador

```
1 def write_codewords_with_error(filename, codeword, max_errors):
2     n = len(codeword)
3     received_word_array_with_errors = []
4     # Convertir codeword [0,1,1,...] -> entero
5     codeword_int = int("".join(map(str, codeword)), 2)
6     # Todas las posiciones posibles donde colocar los errores, por peso
7     #  $C(24,0) + C(24,1) + C(24,2) + C(24,3) + C(24,4) = 12951$ 
8     error_masks = [
9         sum(1 << (n - 1 - pos) for pos in error_positions)
10        for n_errors in range(max_errors + 1)
11        for error_positions in combinations(range(n), n_errors)
12    ]
13    with open(filename, "w") as file:
14        for mask in error_masks:
15            received_word = codeword_int ^ mask
16            received_bits = np.array(
17                [int(bit) for bit in format(received_word, f"0{n}b")],
18                dtype=np.uint16)
19            msg, corrected, uncorrectable = decoder_model.decode(
20                received_bits)
21            received_word_array_with_errors.append(received_bits)
22            file.write(" ".join([
23                format(received_word, f"0{n}b"),
24                "".join(map(str, msg)),
25                format(mask, f"0{n}b") if uncorrectable is False else
26                format(0, f"0{n}b"),
27                str(int(corrected)),
28                str(int(uncorrectable)),
29            ]) + "\n")
30    return np.array(received_word_array_with_errors), msg, error_masks,
31           corrected, uncorrectable, len(error_masks)
32
33 import os
34 import shutil
35 max_errors = 4 # 4 errors limit
36 filename_codeword_with_errors = "outputs/decoder/decoder_top_testing/
37     codeword_with_errors.txt"
38 received_word_array_with_errors, msg, error_masks, corrected, uncorrectable
39 , n_lines = write_codewords_with_error(filename_codeword_with_errors,
40     encoder_output[3838], max_errors)
41 print(f"{n_lines} codewords written to {filename_codeword_with_errors}")
42 # Copia para las secuencias UVM
43 uvm_sequences_dir = "../implem/golay_decoder/decoder/decoder.uvm/sequences"
44 os.makedirs(uvm_sequences_dir, exist_ok=True)
45 uvm_codeword_with_errors = shutil.copy(filename_codeword_with_errors,
46     uvm_sequences_dir)
47 print(f"{n_lines} codewords written to {uvm_codeword_with_errors}")
```

2.2. Clase GaloisField en GF(2)

Para comenzar se instanci6 la clase previamente desarrollada para modelar campos de Galois con parametros $m = 1$ y $P(x) = x + 1$, para testear y corregir posibles issues. La implementaci6n existente no requiri6 modificaciones y se validaron todas sus operaciones exitosamente:

```
Field Closed Succesfully!, 1 Non-Zero Elements
=====
1 -- 0
0 -- -1
=====

ADD:
gf.add(1,1)    = 0
gf.add(1,0)    = 1
gf.add(0,1)    = 1
gf.add(0,0)    = 0

MUL:
gf.mul(1,1)    = 1
gf.mul(1,0)    = 0
gf.mul(0,1)    = 0
gf.mul(0,0)    = 0

INV:
gf.inv(1)      = 1
gf.inv(0)      = Raises: ZeroDivisionError

DIV:
gf.div(1,1)    = 1
gf.div(1,0)    = Raises: ZeroDivisionError
gf.div(0,1)    = 0
gf.div(0,0)    = Raises: ZeroDivisionError

POW:
gf.pow(1,0)    = 1
gf.pow(1,1)    = 1
gf.pow(1,2)    = 1
gf.pow(0,0)    = 1
gf.pow(0,1)    = 0
gf.pow(0,2)    = 0
```

Figura 1: Verificaci6n de las operaciones en GF2.

Una vez verificado el funcionamiento de la clase, se implementaron los metodos para realizar el producto vector-matriz, matriz-matriz, peso de Hamming y algunos metodos auxiliares que se usaron en los modelos del encoder y decoder.

Usando estos nuevos metodos se verificaron las principales propiedades para las matrices del codigo ($GH^T = 0$ y $B^2 = I$):

```

Matrix G (systematic):
=====
[[1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 1 0 0 0 1 1 1 1]
[0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 1 1 1 0 0 1 1 1]
[0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 1 0 1 0 1 0 1 1 1]
[0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 1 1 1 1 1 0 0 0 1 0]
[0 0 0 0 1 0 0 0 0 0 0 0 0 1 1 0 1 1 1 1 0 1 0 0 0 1]
[0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 1 1 1 1 0 0 1 1 0 0]
[0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 1 1 1 1 0 0 1 1 0 0]
[0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 1 1 1 1 0 1]
[0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 1 0 1 1 1 1 1 0]
[0 0 0 0 0 0 0 0 0 1 0 0 0 1 0 0 0 0 1 1 1 1 0 1 1]
[0 0 0 0 0 0 0 0 0 0 1 0 0 1 1 1 1 0 0 1 1 1 0 1 0]
[0 0 0 0 0 0 0 0 0 0 0 1 0 1 1 1 1 0 0 0 1 1 0 1 0]
[0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 0 1 0 1 0 1 0 1]]

Matrix H:
=====
[[1 0 0 1 1 0 0 0 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0]
[0 1 0 0 1 1 1 0 0 1 1 1 1 0 1 0 0 0 0 0 0 0 0 0 0]
[0 0 1 1 0 1 0 1 0 1 0 1 1 1 0 0 1 0 0 0 0 0 0 0 0]
[1 0 1 1 1 1 1 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0]
[1 1 0 1 1 1 0 1 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0]
[0 1 1 1 1 1 0 0 1 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0]
[0 1 0 1 0 0 1 1 1 1 1 0 1 0 0 0 0 0 1 0 0 0 0 0]
[0 0 1 0 1 0 1 1 1 1 1 0 0 0 0 0 0 0 0 1 0 0 0 0]
[1 0 0 0 0 1 1 1 1 0 1 1 0 0 0 0 0 0 0 0 1 0 0 0]
[1 1 1 0 0 1 1 1 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0]
[1 1 1 1 0 0 0 1 1 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0]
[1 1 1 0 1 0 1 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1]]

```

```

Property: G*HT = 0
=====
[[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0]]

Property: B*B = I
=====
[[1 0 0 0 0 0 0 0 0 0 0 0 0 0]
[0 1 0 0 0 0 0 0 0 0 0 0 0 0]
[0 0 1 0 0 0 0 0 0 0 0 0 0 0]
[0 0 0 1 0 0 0 0 0 0 0 0 0 0]
[0 0 0 0 1 0 0 0 0 0 0 0 0 0]
[0 0 0 0 0 1 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 1 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 1 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 1 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 1 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 1 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 1 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 1 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 1]]

```

Figura 2: Matrices y propiedades del código verificadas.

2.3. Modelado del encoder y decoder

A partir de la clase desarrollada previamente para $GF(2)$, se implementaron los modelos correspondientes al encoder y al decoder.

Se desarrolló el modelo del encoder implementando la codificación sistemática mediante producto matricial. La lógica correspondiente fue encapsulada en la clase GolayEncoder, y se verificó la distribución de pesos esperada del código extendido Golay 24,12:

Peso	0	8	12	16	24
Cantidad de palabras	1	759	2576	759	1

```

Golay 24,12 Weight Distribution:
=====
W0      : 1
W8      : 759
W12     : 2576
W16     : 759
W24     : 1

```

Figura 3: Distribución de pesos del código y modelo.

Posteriormente, se desarrolló el modelo del decoder implementando los diferentes casos de la decodificación. La lógica correspondiente fue encapsulada en la clase GolayDecoder.

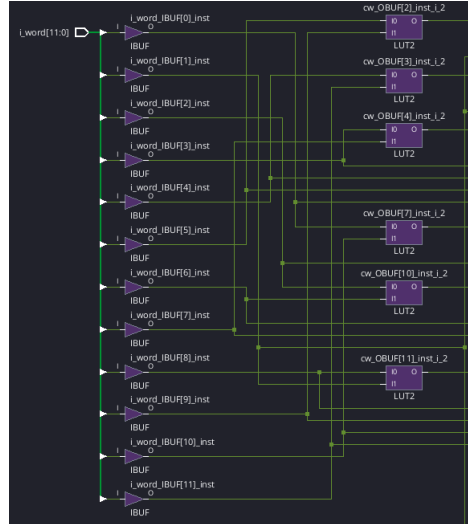
(Parte de AbeliyoPiyo vvvvvv) * caracterizamos el modelo del decoder, iteramos para todas las combinaciones con error de pesos 0 a 4 y todas las codewords, analizamos los casos corregidos y detectados obteniendo: ACA TAMBIEN ME COLGUE XD...

3. Parte C — Implementación RTL

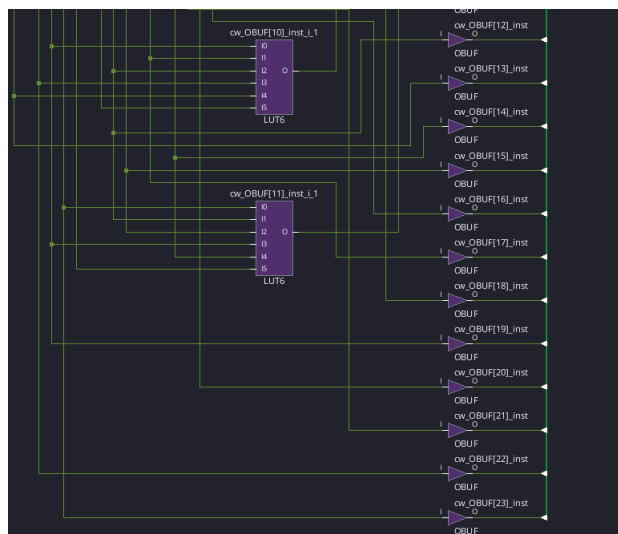
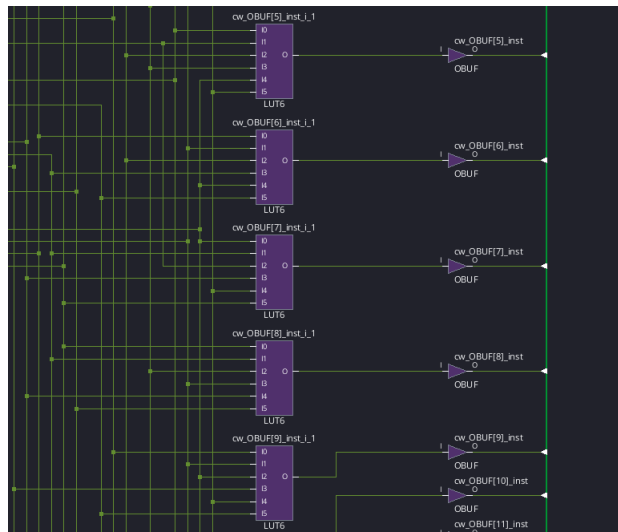
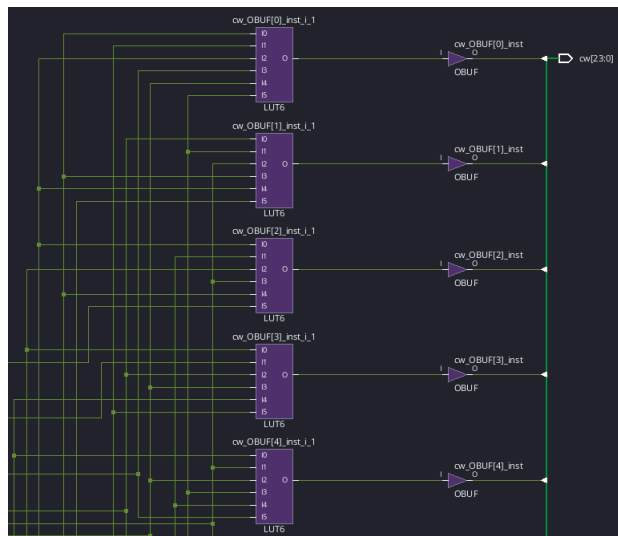
4. Codificador

Se diseñó el módulo de codificación de codewords para realizar la multiplicación de la información recibida con la matriz $\mathbf{G} = I|B$.

Se presenta la implementación del codificador. En la siguiente figura se muestra la primer etapa de buffers y las primeras 6 LUTs del circuito.



Luego se presentan las siguientes 12 LUTs con los buffers de salidas del módulo.



4.1. Validación del encoder

Se realiza la simulación con el objetivo de verificar la generación de las codewords a partir de las 4096 combinaciones posibles de la palabra de 24-bits a codificar. Las posibles combinaciones y las codewords resultantes se obtienen del golden model realizado en software. El formato del archivo es el siguiente:

Columna 1	Columna 2
Información	Codeword

Se muestra la simulación realizada en Vivado:

Name	Value	0.000 ns												20.000 ns												40.000 ns												60.000 ns												80.000 ns												100.000 ns																																																																																			
> i_word[11:0]	000	000												001												002												003												004												005												006												007												008												009												00a												00b											
> cw[23:0]	000000	000000												001ea9												002f1a												0031b3												004e74												0059dd												00616e												007fc7												00887b												0096d2												00a761												0...											
> exp_cw[23:0]	000000	000000												001ea9												002f1a												0031b3												004e74												0059dd												00616e												007fc7												00887b												0096d2												00a761												0...											
> line_num[31:0]	00000001	00000001												00000002												00000003												00000004												00000005												00000006												00000007												00000008												00000009												0000000a												0000000b												0...											
> n_vectors[31:0]	00000000	00000001												00000002												00000003												00000004												00000005												00000006												00000007												00000008												00000009												0000000a												0000000b												...											

Figura 4: Verificación del encoder

Mediante la consola de TCL podemos ver que la implementación en RTL procesa la información correctamente.

```

launch_simulation: Time (s): cpu = 00:00:11 ; elapsed = 00:00:10 .
run -all
-----
golay_encoder: 4096 vectors, 0 failed
RESULT: PASS
-----
$finish called at time : 40960 ns : File "/home/abel/Desktop/desktop

```

4.2. Desarrollo y verificación de módulos comunes

Se comenzo describiendo en Verilog los módulos comunes que serán utilizados posteriormente como parte de los bloques correspondientes al encoder y al decoder.

Una vez implementados, los módulos fueron verificados mediante un testbench sencillo en UVM, con secuencias básicas para validar el funcionamiento de cada uno.

4.3. Analisis: Posible modificacion del decoder

Para el decoder la arquitectura propuesta plantea el cálculo del síndrome y del vector q en etapas diferentes del pipeline. A partir de las propiedades del código, en particular de la condición $B^2 = I$, se analizó una posible modificación de la arquitectura en la cual se calculan el syndrome y q en la primera tapa del pipeline directamente usando el vector recibido r .

Considerando al vector r por sus componentes r_h (alta) y r_l (baja), y como el síndrome:

$$s = r_h \cdot B + r_l$$

Como el vector q se obtiene a partir del síndrome, si aprovechamos la propiedad $B^2 = I$ se puede expresar q directamente en función de los componentes del vector recibido:

$$\begin{aligned}
 q &= s \cdot B \\
 &= (r_h \cdot B + r_l) \cdot B \\
 &= r_h \cdot B^2 + r_l \cdot B \\
 &= r_h \cdot I + r_l \cdot B \\
 &= r_h + r_l \cdot B
 \end{aligned}$$

De esta manera se permite plantear una alternativa donde el síndrome y el vector q serían calculados a partir de r durante la primera etapa del pipeline.

Sin embargo, el análisis de la implementación mostró que la modificación requiere una cantidad adicional de lógica que no resultaba conveniente para la arquitectura propuesta. En particular, se agregan 12 compuertas XOR adicionales, junto con la lógica necesaria para seleccionar el cálculo correspondiente al síndrome o al vector q .

Considerando tanto el incremento logica como un analisis basico de la actividad de switching de los modulos cuando cambia el vector r , se descartó esta alternativa. Por lo tanto, se mantuvo la arquitectura originalmente propuesta, conservando el cálculo del síndrome y del vector q en las etapas de pipeline inicialmente definidas.

4.4. Implementación RTL del encoder

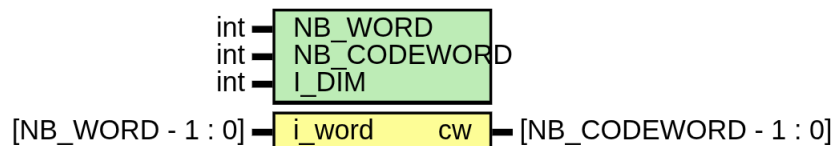
(Parte de AbeliyoPiyo vvvvvv) * RTL DEL ENCODER

4.5. Resumen del encoder

Entity: golay_encoder

• File: golay_encoder.sv

Diagram



Generics

Generic name	Type	Value	Description
NB_WORD	int	12	
NB_CODEWORD	int	24	
I_DIM	int	12	

Ports

Port name	Direction	Type	Description
i_word	input	[NB_WORD - 1 : 0]	
cw	output	[NB_CODEWORD - 1 : 0]	

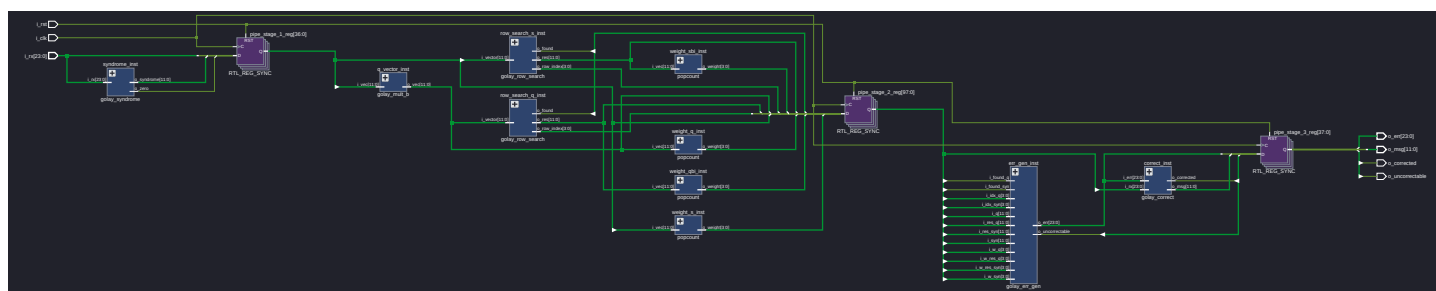
Figura 5: Resumen de la implementación RTL del encoder.

4.6. Implementación RTL del decoder

A partir de los módulos comunes, se describieron bloques para el cálculo del síndrome, del vector q , la búsqueda por filas y la generacion de la máscara de error mediante los diferentes casos en la decodificación.

Cada uno fue validado mediante el testbench UVM, utilizando secuencias de estímulos basicas para verificar su funcionamiento.

Una vez validados los módulos, se incorporaron los registros de pipeline correspondientes para integrar el top del decoder. Quedando la arquitectura final y cada etapa logica:



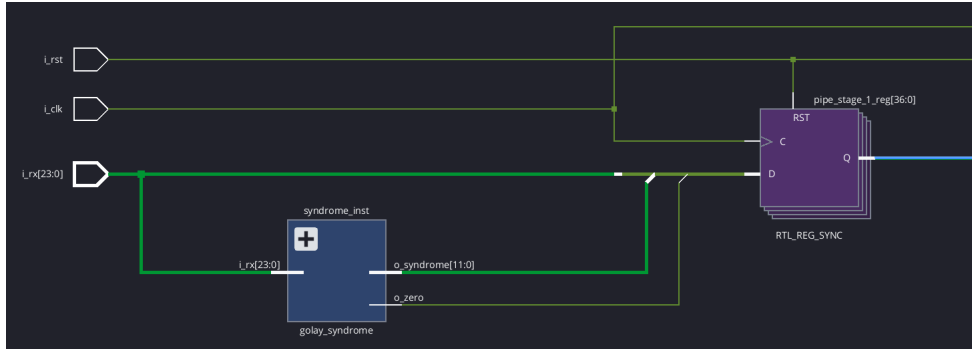


Figura 6: Primera etapa del pipeline del decoder.

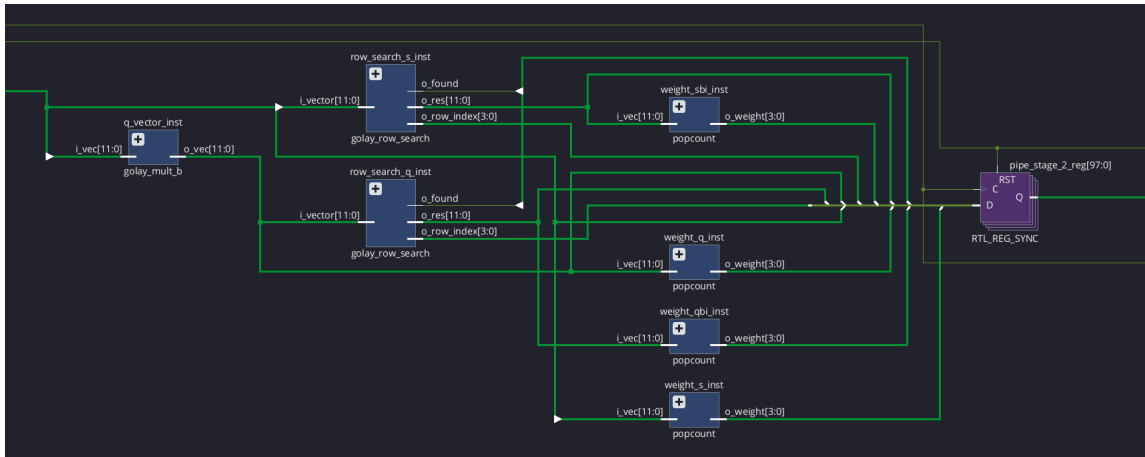


Figura 7: Segunda etapa del pipeline del decoder.

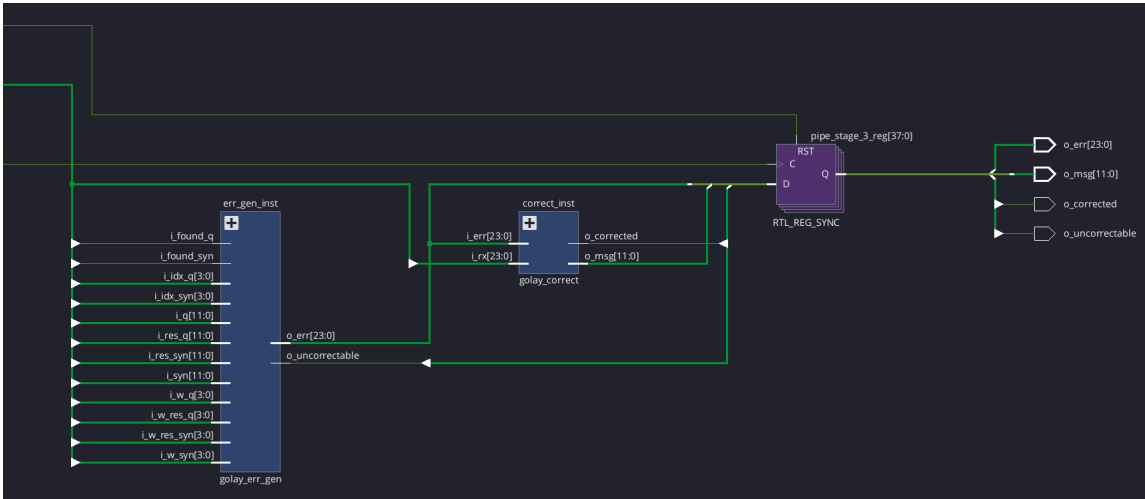


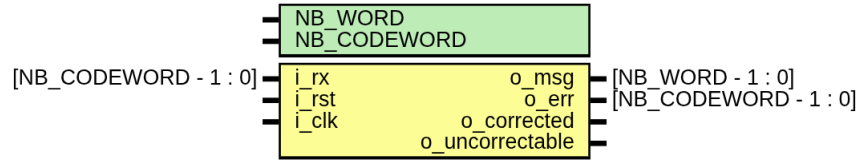
Figura 8: Tercera etapa del pipeline del decoder.

4.7. Resumen del decoder

Entity: decoder

• File: decoder.sv

Diagram



Generics

Generic name	Type	Value	Description
NB_WORD		12	
NB_CODEWORD		24	

Ports

Port name	Direction	Type	Description
o_msg	output	[NB_WORD - 1 : 0]	
o_err	output	[NB_CODEWORD - 1 : 0]	
o_corrected	output		
o_uncorrectable	output		
i_rx	input	[NB_CODEWORD - 1 : 0]	
i_rst	input		
i_clk	input		

Figura 9: Resumen de la implementación RTL del decoder.

4.8. Validacion del decoder

Se realizó su verificación funcional usando el entorno UVM mediante vector matching, con archivos de referencia generados a partir del modelo desarrollado previamente. En los vectores cada línea contiene el vector de entrada *rx* y las salidas esperadas correspondientes al mensaje decodificado, el patrón de error y las flags corrected y uncorrectable.

Como primera verificación funcional, se utilizaron codewords sin errores. En este caso se verificó que el patrón de error sea nulo, y que las señales corrected y uncorrectable permanecieran en bajo durante todo el test. La Figura 10 muestra los resultados obtenidos durante el test:

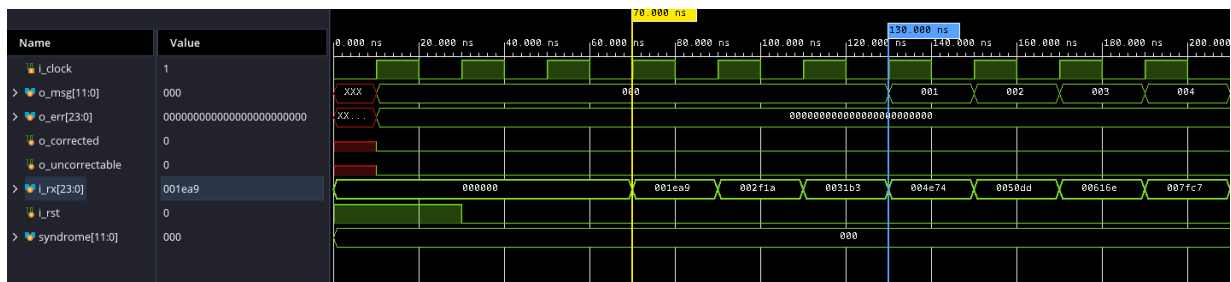


Figura 10: Verificación del decoder utilizando codewords sin errores.

En esta ventana se observa también la latencia de tres ciclos esperada. Sobre el primer cursor, en

hexadecimal los tres dígitos superiores de rx corresponden al mensaje 001h junto con los bits de paridad correspondientes EA9h. Luego de tres ciclos, se obtiene a la salida el mismo mensaje decodificado.

4.9. Correcciones en el decoder

A continuación, se realizó una segunda prueba aplicando en cada codeword todos los patrones de error posibles correspondientes a pesos de error entre cero y cuatro.

Durante la verificación se detectaron algunos errores en el decoder. Los problemas estuvieron relacionados a condiciones en la generación de la máscara de error y al conexionado entre los diferentes módulos.

Por observación y análisis, se identificaron los problemas y se realizaron las correcciones correspondientes. Luego de aplicar las correcciones, se logró validar el funcionamiento del decoder para los casos contemplados en el test.

Se puede ver un waveform general del test donde, muy resumidamente, luego de salir del reset se aplican errores de peso 0 a 3 donde se recupera el mensaje original (EFEh) y se reportan los casos como corrected. Luego se insertan errores de peso 4 hasta el final del test y se reportan como uncorrectable:



Figura 11: Simulación del decoder para los patrones de error evaluados.

Y también se agregó un objeto en el testbench para reportar la cantidad de patrones de error por peso (en formato de array `error_weight[weight]`) y la cantidad de casos corregidos y detectados. Se obtuvieron los valores de la tabla verificada en la PARTE B (Ejercicio 7 Caracterización del decodificador):

Peso de e	0	1	2	3	4
Patrones	1	24	276	2024	10626
Corregidos	1	24	276	2024	0
Detectados	0	0	0	0	10626

Figura 12: Tabla de casos de decodificación según peso de error (modelo).

```

UVM_INFO /home/clifferto/Documents/fec-lab2/implen/golay_decoder/decoder/decoder.uvm/tb_test.svh(29) @ 0: uvm_test_top [TEST] Resetting DUT
UVM_INFO /home/clifferto/Documents/fec-lab2/implen/golay_decoder/decoder/decoder.uvm/tb_virtual_seq.svh(14) @ 30000: reporter@vseq [vseq]
Launching codewords_with_error_seq...

relaunch_sim: Time (s): cpu = 00:04:12 ; elapsed = 00:02:38 . Memory (MB): peak = 11417.199 ; gain = 0.000 ; free physical = 550 ; free virtual = 20019
run all
UVM_INFO /home/clifferto/tools/2025.2/Vivado/data/system_verilog/uvm_1.2/xlnx_uvm_package.sv(19968) @ 259130000: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
-----
Name          Type          Size  Value
-----
report        tb_report    -      @575
error_weight  sa(integral)  5      -
[0]           integral      32     'd1
[1]           integral      32     'd24
[2]           integral      32     'd276
[3]           integral      32     'd2024
[4]           integral      32     'd10626
corrected     sa(integral)  5      -
[0]           integral      32     'd0
[1]           integral      32     'd24
[2]           integral      32     'd276
[3]           integral      32     'd2024
[4]           integral      32     'd0
detected      sa(integral)  5      -
[0]           integral      32     'd0
[1]           integral      32     'd0
[2]           integral      32     'd0
[3]           integral      32     'd0
[4]           integral      32     'd10626
-----
UVM_INFO /home/clifferto/tools/2025.2/Vivado/data/system_verilog/uvm_1.2/xlnx_uvm_package.sv(13673) @ 259130000: reporter [UVM/REPORT/SERVER]
--- UVM Report Summary ---

** Report counts by severity
UVM_INFO : 6
UVM_WARNING : 0
UVM_ERROR : 0
UVM_FATAL : 0
** Report counts by id
[RNTEST] 1
[TEST] 1
[TEST_DONE] 1
[UVM/COMP/NAMECHECK] 1
[UVM/RELNOTES] 1
[vseq] 1

```

Figura 13: Tabla de casos de decodificación según peso de error (RTL).

Finalmente, se verificó el funcionamiento del decoder utilizando las codewords correspondientes a la PARTE A (Ejercicio 3 Decodificación manual). Las tres codewords fueron aplicadas como vectores de entrada y se verificó su correspondiente decodificación. La Figura 14 muestra las señales obtenidas durante esta prueba.

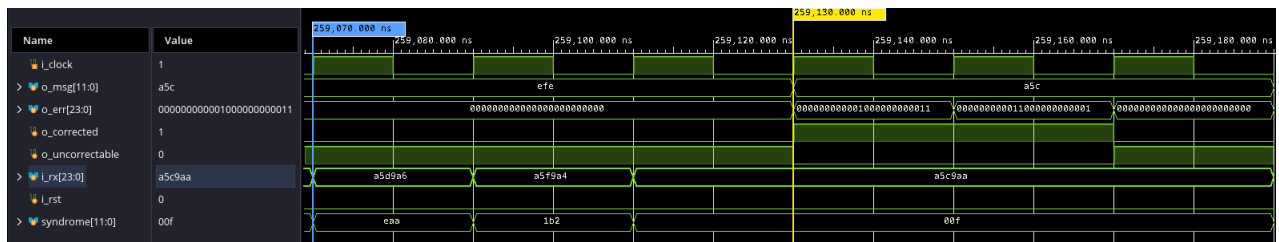
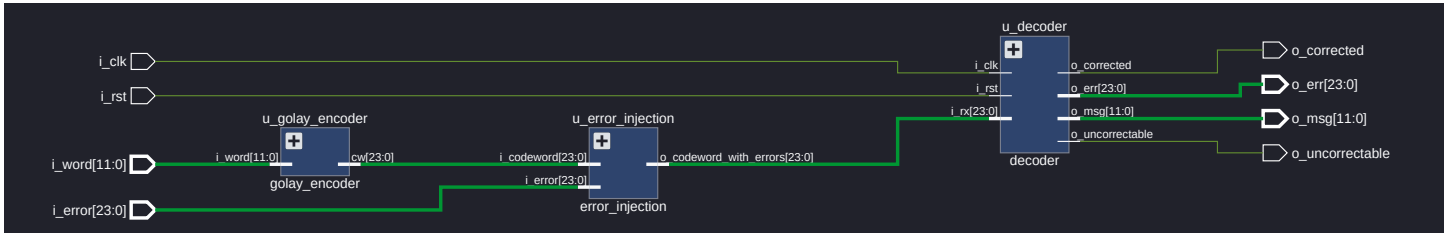


Figura 14: Decodificación de las tres codewords correspondientes a la PARTE A.

4.10. Integración del top level

En esta etapa se realizó la integración de los módulos desarrollados previamente obteniendo el top level del diseño. Para ello, se instanciaron los módulos encoder, decoder e inyector de errores, estableciendo las conexiones necesarias entre ellos de acuerdo con la arquitectura definida.



4.11. Verificación del top level

Una vez integrado el top level, se generaron los vectores necesarios para validar el funcionamiento de la arquitectura final. Cada línea del archivo de vectores contiene, en orden, la palabra a transmitir y el patrón de error a aplicar. Como resultados esperados se incluyen el mensaje decodificado, la máscara de error detectada y las señales corrected y uncorrectable.

La validación se realizó mediante el entorno UVM, utilizando vector matching entre los resultados obtenidos por el diseño RTL y los valores generados previamente a partir del modelo de referencia. La Figura 15 muestra una vista general de la simulación correspondiente al test, donde se aplican patrones de error de peso menor o igual a tres, seguida por una región en la que se aplican patrones de error incorregibles.

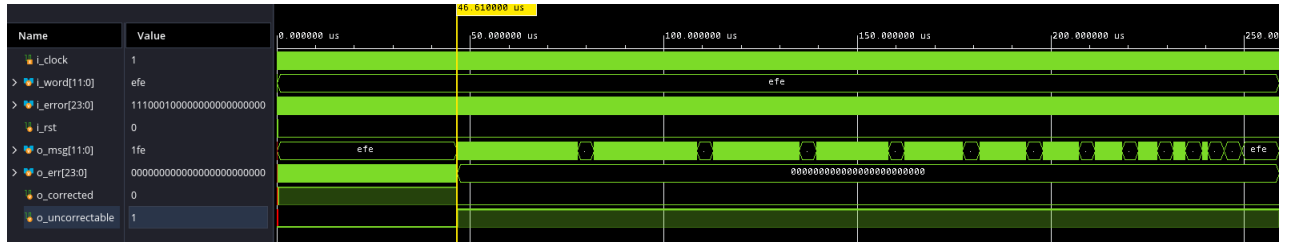


Figura 15: Waveform general correspondiente a la verificación del top level.

Al salir del reset se aplica la palabra EFEh sin ningún patrón de error, luego de tres ciclos se obtiene el mensaje decodificado y el patrón de error detectado es nulo. Este comportamiento se muestra en la Figura 16.

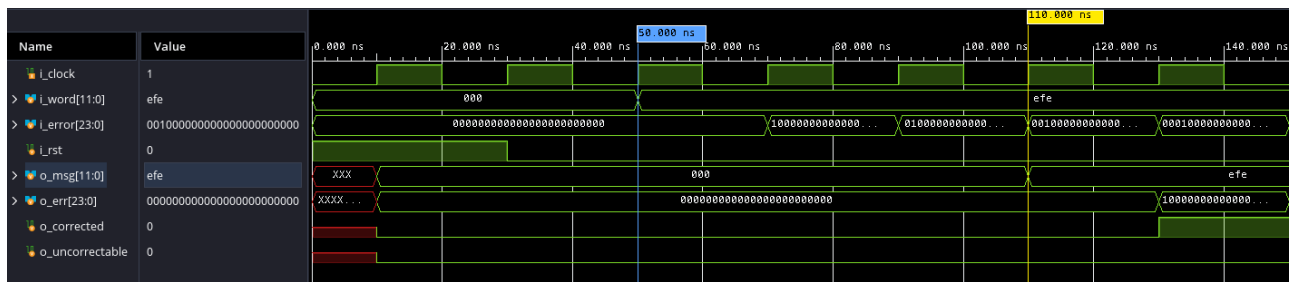


Figura 16: Decodificación sin errores y verificación de la latencia.

A continuación, se incrementó el peso del patrón de error aplicado. En la Figura 17 se muestra la transición desde un patrón de error de peso uno hacia uno de peso dos.

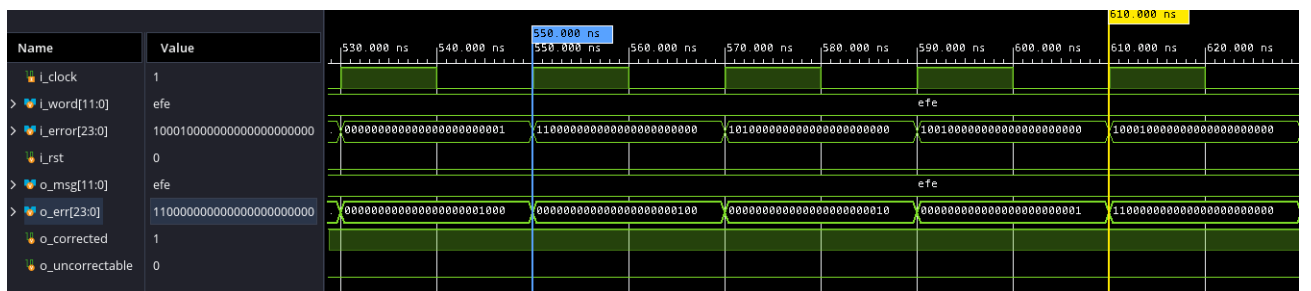


Figura 17: Transición entre patrones de error de peso uno y dos.

La Figura 18 muestra la transición desde un patrón de error de peso dos hacia uno de peso tres.

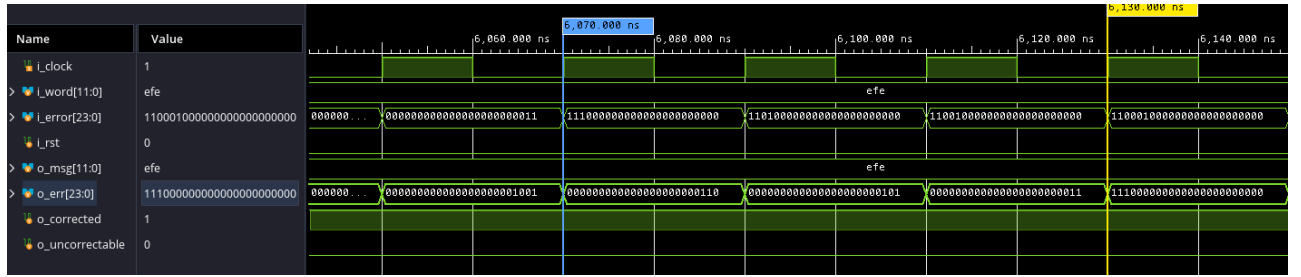


Figura 18: Transición entre patrones de error de peso dos y tres.

Finalmente, en la Figura 19 se observa la transición hacia un patrón de error de peso cuatro. A partir de este punto, la señal uncorrectable permanece en alto, indicando que el patrón de error aplicado no puede ser corregido por el decoder. En esta condición, las salidas correspondientes al mensaje y al patrón de error detectado se consideran inválidas.

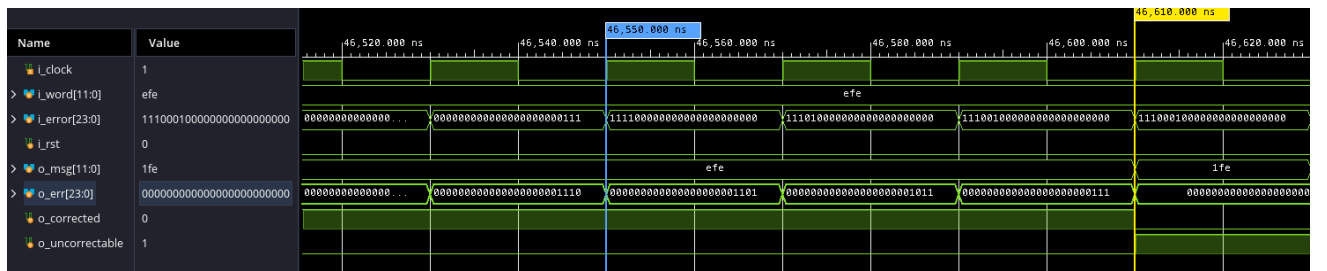


Figura 19: Transición entre patrones de error corregibles e incorregibles.